



COLLECTED WISDOM, EMPOWERED SOFTWARE: YOUR ESSENTIAL ARMORY

Project Title : Offline First Smart Library Management App

Client: Variant Products For Targeted Libraries

Project lead developer Info:

Name: Natinael Samuel (Bsc in Software Engineering)

Phone: +251904161978

Email: afritioalberts1216@gmail.com

Portfolio: <https://natinael-samuel.netlify.app>

01 December 2025

Addis Abeba, Ethiopia

Documentation Overview

System Identity: Offline-First Smart Library Management System

Project Code: church-library-app

Production Brands: Otona St. Mary Library Management / Fayda Smart Library Management

Version: 1.1 (Production Candidate)

Author: DiguwaSoft

Architect: Natinael Samuel

This documentation provides a comprehensive technical breakdown of the architectural decisions, implementation details, and security protocols established during the Phase 1 development cycle. It serves as the primary source of truth for the system's engineering logic, ensuring that the offline-first capability and data integrity measures are maintained throughout the product's lifecycle.

Table of Contents

Documentation Overview.....	1
1. Introduction.....	6
1.1 Purpose of the Document.....	7
1.3 Scope and Phase Coverage.....	7
1.4 Document Structure.....	8
2. Product Overview.....	8
2.1 System Description.....	8
2.2 Offline-First Design Rationale.....	9
Core Principles.....	9
2.3 Production Variants (Otona vs. Fayda).....	10
2.3.1 Otona St. Mary Library Management App.....	10
2.3.2 Fayda Smart Library Management.....	10
2.3.3 Shared Core System.....	10
2.4 Commercial Deployment Model.....	10
2.4.1 Sales Models.....	10
2.4.2 Hosting and Technical Management.....	11
2.4.3 Long-Term Maintainability.....	11
3. System Architecture Overview.....	11
3.1 High-Level Application Architecture.....	11
3.1.1 Presentation Layer (Mobile Application).....	12
3.1.2 Local Data Layer (SQLite).....	12
3.1.3 Synchronization Layer (Commit-Based Sync Engine).....	12
3.1.4 Remote Backend Layer (Supabase).....	12
3.2 Offline vs. Online Responsibilities.....	12
3.2.1 Offline Responsibilities (Primary Execution Path).....	12
3.2.2 Online Responsibilities (Secondary, Deferred).....	13
3.3 Trust Boundaries.....	13
3.4 Device-Centric Design.....	13
3.4.1 Benefits of Device-Centricity.....	14

4. Technology Stack.....	14
4.1 Frontend Framework.....	14
4.1.1 Framework Selection.....	14
4.1.2 Rationale.....	14
4.2 Local Storage (SQLite).....	15
4.2.1 Database Engine.....	15
4.2.2 Role in the System.....	15
4.3 Remote Backend (Supabase).....	15
4.3.1 Backend Platform.....	15
4.3.2 Responsibilities & Deployment.....	16
4.4 Sync Infrastructure.....	16
4.4.1 Sync Model.....	16
4.5 Environment Configuration (.env).....	16
4.5.1 Configuration Strategy.....	16
4.5.2 Key Variables & Rationale.....	17
5. Project File Structure.....	17
5.1 Application Root Structure.....	17
5.1.1 Root-Level Overview.....	17
5.1.2 Structural Principles.....	18
5.2 Authentication Module.....	18
5.2.1 Location.....	18
5.2.2 Responsibilities.....	18
5.2.3 Supporting Libraries.....	18
5.3 Attendance Module.....	19
5.3.1 Location.....	19
5.3.2 Responsibilities.....	19
5.3.3 Related Utilities.....	19
5.4 Database Access Layer.....	19
5.4.1 Location.....	19
5.4.2 Design Approach.....	19
5.5 Sync Engine.....	20
5.5.1 Location.....	20
5.5.2 Sync Model.....	20

6. Database Architecture.....	20
6.1 Local SQLite Schema Overview.....	20
6.1.1 Core Tables.....	20
6.2 Supabase Schema Overview.....	24
6.2.1 Schema Parity & Key Differences.....	24
6.3 Data Ownership Rules.....	25
6.3.1 Authority Model.....	25
6.3.2 Conflict Resolution.....	25
6.4 One-Institution-One-Database Model.....	25
6.4.1 Deployment Model.....	25
6.4.2 Sales & Branding Alignment.....	25
7. Authentication & Authorization.....	25
7.1 Role Definitions (Admin vs. Librarian).....	25
7.2 Device Binding.....	26
7.3 PIN Security Model.....	26
8. Shift Management System.....	26
8.1 Shift Definition & Validation.....	27
8.2 Auto Logout Enforcement.....	27
9. Attendance & Verification.....	27
9.1 Attendance Philosophy.....	27
9.2 High Attendance Verification (Phase 1.18).....	27
9.3 Book-Scan Verification Flow.....	28
10. Scanner Integration.....	28
10.1 Camera Permissions & Barcode Standards.....	28
10.2 Scan Routing Architecture.....	28
11. Sync Engine.....	28
11.1 Commit-Based Sync Model.....	29
11.2 sync-push / sync-pull Workflows.....	29
Sync Push (Local → Supabase).....	29
Sync Pull (Supabase → Local).....	29
12. Admin Dashboards.....	30
12.1 Attendance Dashboard Logic.....	30
12.2 Reconciliation Logic.....	30

13. Security & Safety.....	31
13.1 Error Handling & Safety Guards.....	31
13.2 Security Considerations.....	31
Offline Threat Model & Mitigations.....	31
Privacy & Institutional Safety.....	31
14. Implementation Report (Phase 1.0 → Phase 1.18.2).....	31
14.1 Phase-by-Phase Summary.....	32
14.2 Major Design Decisions.....	33
15. Maintainability & Future Phases.....	34
15.1 Extension Guidelines.....	34
15.2 Phase 2 Readiness.....	34
16. Conclusion.....	34
Appendix.....	36
Appendix A: Environment Configuration (.env Variables).....	36
A.1 Core Environment Variables.....	36
A.2 Attendance & Shift Control.....	37
A.3 Branding & UI.....	37
Appendix B: Database Tables (Reference Overview).....	37
B.1 Key SQLite Tables.....	37
Appendix C: Commit & Sync Infrastructure.....	38
C.1 commits Table.....	38
C.2 Sync Flow Reference.....	38
Appendix D: Reference Files & Directories.....	38
D.1 Critical Application Paths.....	38
Appendix E: Terminology Reference.....	38
Appendix F: Ownership & Maintenance.....	39

1. Introduction

This document presents the official engineering and implementation documentation for the **Offline-First Smart Library Management System**, a production-grade software platform designed, developed, and maintained by **DiguwaSoft**.

The system was initially developed under the internal project name church-library-app and has since evolved into a stable, commercially deployable platform released under two distinct production brands:

1. **Otona St. Mary Library Management App** — Optimized for religious institution libraries.
2. **Fayda Smart Library Management** — Tailored for schools, colleges, and public libraries.

Both products share a single, unified core system, differentiated only by branding assets and deployment configuration. The underlying architecture, logic, data models, and operational guarantees remain consistent across all deployments. This documentation formally captures the architectural design, implementation details, operational rules, and phased development history of the system, covering **Phase 1.0 through Phase 1.18.2**.

1.1 Purpose of the Document

The purpose of this document is to serve as:

- **A Technical Reference Manual:** Providing precise explanations of system architecture, modules, data flow, and implementation decisions.
- **An Internal Engineering Record:** Preserving the technical history of the system, including design rationale, constraints, and phased evolution, to support long-term maintainability and controlled growth.
- **A Commercial-Grade System Specification:** Enabling DiguwaSoft to confidently sell, deploy, maintain, and extend the system for multiple institutions while ensuring architectural consistency and operational integrity.

This document is not a user manual. It is intended to describe how the system is built, why it was designed this way, and how it should be maintained or extended.

1.2 Intended Audience

This documentation is primarily intended for:

- **Software Developers and Engineers:** Including future maintainers, contractors, and internal DiguwaSoft staff.
- **System Architects and Technical Decision-Makers:** Responsible for extending the system or evaluating architectural soundness.
- **DiguwaSoft Management:** Serving as a long-term reference for system evolution and onboarding new technical staff.

A working knowledge of TypeScript, React Native (Expo), SQLite, Supabase, and distributed synchronization concepts is assumed.

1.3 Scope and Phase Coverage

This document covers the system implementation from **Phase 1.0 through Phase 1.18.2**. The scope includes:

- Offline-first system architecture.
- Authentication, role enforcement, and device binding.
- Shift scheduling, enforcement, and auto-logout logic.
- Attendance and High Attendance Verification (HAV) protocols.
- Barcode-based scanning workflows and routing.
- Local SQLite and remote Supabase data models.
- Commit-based synchronization (Push/Pull) mechanisms.
- Admin dashboards and data reconciliation.

Future phases beyond Phase 1.18.2 are intentionally excluded and will be addressed in subsequent documentation revisions.

1.4 Document Structure

The document is organized into structured sections that progressively describe the system from high-level concepts to detailed implementation specifics.

- **Foundation:** Introduction of product context, architecture, and the technical stack.
- **Core Logic:** Detailed breakdown of subsystems such as authentication, attendance, scanning, and synchronization.

- **Management & Security:** Explanation of administrative dashboards and security guards.
- **History & Evolution:** A formal implementation report covering completed phases and guidance for future expansion.

All explanations are presented using structured textual descriptions, in accordance with the documentation standards defined for this project.

2. Product Overview

The **Offline-First Smart Library Management System** is a robust, production-grade software platform designed to manage library operations in environments where network reliability cannot be guaranteed. The system prioritizes local availability, data integrity, and operational continuity, while still supporting centralized synchronization and administrative oversight.

Developed by **DiguwaSoft**, the platform targets institutions that require secure, auditable, and scalable library management without dependency on continuous internet connectivity.

2.1 System Description

The system provides a complete end-to-end solution for managing:

- Library users (patrons) and staff (librarians/administrators)
- Book inventory and circulation (borrowing and return transactions)
- Attendance and shift enforcement
- Administrative reporting and auditing
- Secure device-bound authentication and offline data synchronization

The application is implemented as a mobile-first React Native application, backed by a local SQLite database for offline operations and a Supabase-hosted backend for centralized data persistence and synchronization.

2.2 Offline-First Design Rationale

The system adopts an offline-first architectural approach by design, rather than as an afterthought.

Rationale

Target deployment environments—such as churches, schools, and public institutions—often experience:

1. Intermittent or unreliable internet connectivity.
2. Power or infrastructure instability.
3. Limited on-site technical support.

Core Principles

- **Local-First Writes:** All operational actions (borrowing, returning, attendance, scans) are written immediately to the local SQLite database.
- **Deferred Synchronization:** Changes are recorded as structured commits and synchronized with the remote backend when connectivity is available.
- **Operational Safety:** No core workflow depends on real-time network validation.

2.3 Production Variants (Otona vs. Fayda)

Although developed as a single core system, the platform is deployed under two distinct production variants to address different institutional contexts.

2.3.1 Otona St. Mary Library Management App

- **Target Market:** Church-based libraries.
- **Branding:** Dedicated logos, custom splash screens, and a church-aligned visual identity.
- **Functional Scope:** Identical to the core system.

2.3.2 Fayda Smart Library Management

- *Target Market**: Schools, colleges, and public libraries.
- **Branding**: Distinct logos, independent splash screens, and an institution-neutral visual identity.
- **Functional Scope**: Fully identical to the Otona variant.

2.3.3 Shared Core System

Both variants share a single core codebase and database schema. This strategy ensures maintainability, reduces divergence risk, and allows DiguwaSoft to support multiple markets efficiently through branding assets and deployment configuration alone.

2.4 Commercial Deployment Model

The system is commercially deployed and managed exclusively by DiguwaSoft.

2.4.1 Sales Models

The platform is offered via institution-specific licensing or subscription models, determined by direct agreement with the client.

2.4.2 Hosting and Technical Management

For every client institution, DiguwaSoft provisions a dedicated Supabase database. This guarantees:

- **Data Isolation**: Strong tenant isolation between institutions.
- **Managed Operations**: Hosting, updates, and maintenance are handled centrally by DiguwaSoft.
- **Controlled Upgrades**: Clients receive stable, managed updates without technical burden.

2.4.3 Long-Term Maintainability

By retaining technical ownership, DiguwaSoft ensures consistent quality standards and security patches are applied across all institutional deployments.

3. System Architecture Overview

The **Offline-First Smart Library Management System** is architected as a device-centric, locally authoritative application with deferred centralized synchronization. The architecture is intentionally designed to ensure operational continuity, security, and data integrity in environments with unreliable connectivity.

Rather than relying on a traditional client–server real-time model, the system follows a **local-first execution model**, where the client device is the primary execution authority during daily operations.

3.1 High-Level Application Architecture

The system consists of the following architectural layers:

3.1.1 Presentation Layer (Mobile Application)

- Implemented using **React Native with Expo**.
- Provides interfaces for Authentication, Attendance, Book Scanning, and Admin views.
- Operates entirely offline once installed, integrating directly with local storage and device hardware (camera).

3.1.2 Local Data Layer (SQLite)

- An embedded structured SQLite database on the device.
- Stores Users, Librarians, Books, Transactions, Attendance records, and Commit logs.
- All write operations are executed locally first, guaranteeing immediate consistency.

3.1.3 Synchronization Layer (Commit-Based Sync Engine)

- Tracks local changes using structured commit records (Inserts, Updates, Deletes).
- Supports resumable, idempotent synchronization.
- Decouples daily operations from network availability.

3.1.4 Remote Backend Layer (Supabase)

- Acts as the authoritative centralized datastore for long-term persistence.
- Receives synchronized commits and provides administrative oversight.
- Architecture follows a **One-Institution-One-Database** model.

3.2 Offline vs. Online Responsibilities

A strict separation of duties ensures that core workflows are never blocked by network state.

3.2.1 Offline Responsibilities (Primary Execution Path)

- Authentication using locally stored, device-bound credentials.
- Shift validation and enforcement.
- Attendance verification and barcode-driven transactions.
- Commit generation for future synchronization.

3.2.2 Online Responsibilities (Secondary, Deferred)

- Persisting synchronized data for institutional records.
- Administrative reporting and cross-device reconciliation.
- Backups and disaster recovery.

3.3 Trust Boundaries

The system enforces multiple boundaries to ensure auditability and security.

- **3.3.1 Device Trust Boundary:** Each librarian account is bound to a specific physical device. Accounts cannot be used on unauthorized hardware without administrative intervention.
- **3.3.2 Role-Based Trust Boundary:** Local enforcement ensures Librarians are restricted to operational workflows while Administrators are restricted to oversight.
- **3.3.3 Time and Shift Boundary:** Access is strictly limited to scheduled shift windows. Shift expiration triggers automatic, forced logouts locally.
- **3.3.4 Data Synchronization Boundary:** Local data is tentatively authoritative for immediate operations; remote data is eventually authoritative for historical records.

3.4 Device-Centric Design

The system treats the device as a secure operational unit. Because devices are physically located within the library, physical access acts as an implicit security layer.

3.4.1 Benefits of Device-Centricity

- **Reduced Network Risk:** Authentication does not require a round-trip to a server.
- **Physical Presence:** Camera-based scanning and device binding serve as indicators of actual on-site presence.
- **Operational Reliability:** Guarantees that librarians can perform their duties regardless of the external infrastructure's state.

4. Technology Stack

The **Offline-First Smart Library Management System** is implemented using a technology stack optimized for offline reliability, cross-platform deployment, and long-term maintainability. The stack prioritizes deterministic local execution and controlled synchronization over real-time dependency on remote services.

4.1 Frontend Framework

4.1.1 Framework Selection

- **React Native (Expo):** Used for cross-platform mobile deployment with an Android-first focus.
- **TypeScript:** Enforced for type safety and to minimize runtime errors across complex data models.
- **Expo Router:** Utilized for file-system-based navigation and deep linking capabilities.

4.1.2 Rationale

React Native with Expo was selected for its mature ecosystem regarding camera and storage APIs, which are critical for barcode scanning and SQLite management. This choice reduces build complexity and ensures that a single engineering team can maintain both the UI and the underlying logic efficiently.

4.2 Local Storage (SQLite)

4.2.1 Database Engine

- **SQLite (embedded)**: Accessed via the expo-sqlite library.

4.2.2 Role in the System

SQLite is the primary operational datastore. It is treated as the **source of truth** for all daily workflows. It stores:

- Patrons, Librarians, and Inventory (Books).
- Transaction history (Borrowing/Returns).
- Shift and Attendance records.
- **Commit Logs**: Structural records representing every local mutation intended for the backend.

4.3 Remote Backend (Supabase)

4.3.1 Backend Platform

- **Supabase (PostgreSQL-based)**: Provides the centralized data persistence layer.

4.3.2 Responsibilities & Deployment

Supabase handles institutional data aggregation, administrative reporting, and disaster recovery. Under the **One-Institution-One-Database** model, each client institution is isolated in its own Supabase project. This ensures maximum data privacy and simplifies compliance with data governance standards.

4.4 Sync Infrastructure

4.4.1 Sync Model

The system employs a **Commit-Based Sync Model**. Rather than performing direct table-to-table replication, the application logs every local write as a "Commit."

- **Operation Types:** Insert, Update, Delete.
- **Characteristics:** Asynchronous, Idempotent, and Resumable.

This model allows librarians to continue working during network outages; the system simply queues the commits and pushes them to Supabase once a stable connection is detected.

4.5 Environment Configuration (.env)

4.5.1 Configuration Strategy

Business policies and feature flags are managed via environment variables (`process.env.EXPO_PUBLIC_*`), which are injected at build time.

4.5.2 Key Variables & Rationale

Configuration includes thresholds for **High Attendance Verification** (e.g., number of books to scan) and backend connection parameters. This approach decouples business policy from the source code, allowing DiguwaSoft to customize the application behavior for different institutions (e.g., Otona vs. Fayda) without modifying the core logic.

5. Project File Structure

5.1 Application Root Structure

The application follows a modular, responsibility-driven directory structure, aligned with Expo Router conventions and offline-first architectural requirements.

5.1.1 Root-Level Overview

church-library-app/

```
|— app/           # Application routes (Expo Router)
| |— (tabs)/      # Main application tab navigation
| |— auth/        # Login and PIN management screens
| |— attendance/  # Verification and scanning workflows
| |— admin/       # Local administrative dashboards
|— db/           # Local database access layer
| |— queries/     # Domain-specific SQL query functions
| |— sqlite.ts    # Database initialization and schema
| |— commits.ts   # Sync-log management
|— lib/         # Core libraries and session logic
| |— authUtils.ts # Security and hashing helpers
| |— session.ts   # User session state
| |— shiftSession.ts # Shift-specific logic and auto-logout
|— utils/       # Shared utilities and guards
|— types/       # TypeScript interfaces and types
|— assets/      # Images, logos, and branding assets
|— constants/   # Static constants and theme definitions
|— .env         # Environment-specific configuration
|— package.json # Dependencies and scripts
```

5.1.2 Structural Principles

- **Route-driven UI:** UI screens map directly to filesystem routes.
- **Local-first logic:** Database access and validation occur locally.
- **Separation of concerns:** UI, data access, session logic, and sync are isolated.
- **Extensibility:** New modules can be introduced without refactoring existing flows.

5.2 Authentication Module

5.2.1 Location

```
app/auth/  
├── login.tsx      # Primary authentication entry point  
└── change-pin.tsx # Security management for librarians
```

5.2.2 Responsibilities

The authentication module is responsible for local credential verification, device binding enforcement, role-based access control, session initialization, and conditional flow routing (e.g., attendance verification).

5.2.3 Supporting Libraries

```
lib/  
├── authUtils.ts    # PIN hashing and cryptographic helpers  
├── session.ts      # Global user and role state management  
└── shiftSession.ts # Temporal access and auto-logout logic
```

These libraries handle PIN hashing, session persistence, and shift-bound auto-logout enforcement.

5.3 Attendance Module

5.3.1 Location

```
app/attendance/  
├── verify.tsx      # High Attendance Verification entry  
└── scan.tsx        # Barcode scanning logic for presence
```

5.3.2 Responsibilities

The attendance module enforces high-trust attendance verification for librarians by redirecting post-login when enabled, requiring physical book scans, preventing duplicate scans, and enforcing shift validity.

5.3.3 Related Utilities

utils/

└─ shift.ts # Shift calculation and overlap detection

db/queries/

└─ attendance.ts # SQL for attendance and HAV logging

These components handle shift detection, validation, and implicit attendance recording.

5.4 Database Access Layer

5.4.1 Location

db/

- sqlite.ts, books.ts, users.ts, transactions.ts, commits.ts
- queries/librarians.ts, attendance.ts, meta.ts

5.4.2 Design Approach

The database layer follows a "one file per domain" rule, with no UI logic inside database files. All SQL is centralized and version-controlled with explicit read/write functions.

5.5 Sync Engine

5.5.1 Location

db/commits.ts

lib/sync.ts (planned / extendable)

5.5.2 Sync Model

The sync engine operates using a commit log pattern where every local mutation generates a queued commit. The sync process transmits these commits to Supabase while tracking status per commit. It is designed to be offline-safe and failure-isolated.

6. Database Architecture

6.1 Local SQLite Schema Overview

The local SQLite database is the primary source of truth during application runtime. All user actions, validations, and business rules are executed locally first, without network dependency.

6.1.1 Core Tables

Table: users	Type	Description
fayda_id	TEXT (PK)	National / institutional ID
name	TEXT	Full name
phone	TEXT	Contact number
photo_uri	TEXT	Cloudinary or local image reference
created_at	TIMESTAMP	Record creation timestamp

Table: books	Type	Description
book_code	TEXT (PK)	Barcode / QR code identifier
title	TEXT	Book title
author	TEXT	Author name
copies	INTEGER	Total available copies

Table: transactions	Type	Description
tx_id	TEXT (PK)	Transaction UUID
fayda_id	TEXT	Borrower reference
book_code	TEXT	Borrowed book reference
borrowed_at	TIMESTAMP	Time of borrowing
returned_at	TIMESTAMP	Time of return (NULL if active)

sync_status	TEXT	pending / synced
-------------	------	------------------

Table: librarians	Type	Description
username	TEXT (PK)	Login identifier
pin_hash	TEXT	Argon2 or SHA-256 secure hash
role	TEXT	admin / librarian
device_id	TEXT	Bound hardware identifier

Table: shift_attendance	Type	Description
id	INTEGER (PK)	Unique record ID
shift_id	INTEGER	Shift reference
librarian_username	TEXT	Librarian reference
clock_in	TIMESTAMP	Attendance timestamp

status	TEXT	in_progress / completed
--------	------	-------------------------

Table: commits	Type	Description
id	INTEGER (PK)	Local commit ID
type	TEXT	insert / update / delete
table_name	TEXT	Target remote table
payload	JSON	Mutation data payload
synced	BOOLEAN	Sync completion state

6.2 Supabase Schema Overview

Supabase serves as the remote authoritative mirror of institutional data.

6.2.1 Schema Parity & Key Differences

The Supabase schema mirrors the local schema to ensure data consistency, utilizing identical primary keys and table names.

Aspect	Local SQLite	Supabase
Authority	Operational (Runtime)	Persistence (Backup/Admin)
Latency	Zero	Dependent on Network
Mutations	Immediate	Commit-based reconciliation
Security	Local App Logic	Row-Level Security (RLS)

6.3 Data Ownership Rules

6.3.1 Authority Model

The system follows a "Local-Wins" authority model. Transactions, Attendance, and Inventory mutations are mastered locally. Supabase acts as a passive recipient of these commits.

6.3.2 Conflict Resolution

- **Idempotency:** Duplicate commits are ignored by the backend.
- **Integrity:** Supabase rejects transitions that violate relational integrity (e.g., returning a book that wasn't borrowed).
- **Logging:** Conflicts are logged for administrative review.

6.4 One-Institution-One-Database Model

6.4.1 Deployment Model

Each institution (e.g., Otona St. Mary or a specific College) receives a fully isolated Supabase project. This guarantees **strong data isolation**, simplified compliance with privacy laws, and predictable performance.

6.4.2 Sales & Branding Alignment

The same core database architecture supports all production variants. Branding (Otona vs. Fayda) is applied exclusively at the UI and asset layer, ensuring the engineering core remains unified and maintainable across all commercial deployments.

7. Authentication & Authorization

7.1 Role Definitions (Admin vs. Librarian)

The system enforces strict role-based access control (RBAC) to ensure operational integrity.

- **Admin:** Possesses full system authority. Responsible for managing librarian accounts, reviewing synchronized attendance logs, and overseeing institutional data. Admins are exempt from shift enforcement and high attendance verification.
- **Librarian:** The primary operational role. Librarians are subject to strict shift constraints, mandatory attendance verification (if enabled), and device-bound login restrictions.

7.2 Device Binding

Each librarian account is cryptographically bound to a single physical device.

- **Binding Logic:** The unique `device_id` is captured during the first successful login and stored both in the local librarians table and the remote Supabase instance.
- **Enforcement:** Subsequent login attempts verify the current hardware ID against the stored ID. A mismatch results in an immediate denial of access, preventing credential sharing and "ghost" attendance.

7.3 PIN Security Model

To optimize for fast-paced institutional environments, the system utilizes a secure PIN-based model.

- **Storage:** PINs are never stored in plaintext; they are stored as salted hashes using local SQLite.
- **Policy:** PIN rotation is enforced upon the first login to ensure that default credentials are replaced immediately.

8. Shift Management System

8.1 Shift Definition & Validation

Shifts represent the temporal authority for a librarian. Any operational action (borrowing, returning, or scanning) is wrapped in an `isInsideShift()` validation check.

- **Local Enforcement:** Validation occurs entirely on the device, checking the current system time against the active shift window stored in the local database.

8.2 Auto Logout Enforcement

The system maintains a "zero-leakage" policy regarding access. A forced, immediate logout is triggered if:

1. The current shift expires.
2. High Attendance Verification (HAV) is cancelled or fails.
3. A role violation is detected during a session.

9. Attendance & Verification

9.1 Attendance Philosophy

Attendance is a **trust gate**, not merely a record. It is a functional prerequisite that proves a librarian is physically present at the library before they can modify the database.

9.2 High Attendance Verification (Phase 1.18)

Introduced in Phase 1.18, HAV adds a physical layer of security.

- **Trigger:** Activated via EXPO_PUBLIC_HIGH_ATTENDANCE_VERIFICATION=true.
- **Requirement:** Librarians must scan a configurable number of physical books (e.g., 3 books) to prove presence.

9.3 Book-Scan Verification Flow

1. **Login:** Librarian authenticates.
2. **Redirect:** System intercepts the session and routes to the Verification Screen.
3. **Scan:** The librarian must scan \$N\$ distinct, valid book barcodes from the library's physical inventory.
4. **Completion:** Only after the \$N\$th valid scan is the attendance marked as in_progress and the librarian granted access to the dashboard.

- **Rationale:** Unlike GPS (which can be spoofed) or Biometrics (which require specialized hardware), book scanning leverages the library's existing assets to provide irrefutable proof of presence.

10. Scanner Integration

10.1 Camera Permissions & Barcode Standards

The system utilizes the expo-camera library, supporting industry-standard formats including QR, Code128, and EAN-13. Permissions are requested just-in-time and handled gracefully to prevent app crashes.

10.2 Scan Routing Architecture

The scanner is designed as a decoupled, reusable utility. It accepts a `returnTo` route parameter. Once a valid barcode is captured, it passes the data back to the calling module (e.g., Borrowing, Returning, or Attendance) for context-specific validation. This ensures a single, high-performance scanning implementation is used throughout the entire application.

11. Sync Engine

11.1 Commit-Based Sync Model

The system implements a commit-based synchronization model designed specifically for offline-first reliability.

Core Principle: No direct remote writes occur during operational workflows. Every mutation to local data generates a persistent commit record in the local commits table.

A commit record includes:

- **Operation Type:** (INSERT, UPDATE, DELETE).
- **Target Table:** The specific Supabase table to be updated.
- **Payload:** A JSON object containing the mutation data.
- **Metadata:** Timestamp, Librarian identity, and Device ID.

This durable change log guarantees that no operation is lost even during extended offline periods.

11.2 sync-push / sync-pull Workflows

Sync Push (Local → Supabase)

Pending commits are batched and transmitted in chronological order.

- **Idempotency:** Every commit is independently verifiable. If a network interruption occurs, the system can safely retry the same commit without creating duplicate records.
- **Acknowledgement:** A commit is only marked as synced=true in the local database after the Supabase backend acknowledges successful persistence.

Sync Pull (Supabase → Local)

The system periodically fetches authoritative updates from Supabase (e.g., new book inventory or user registrations) and merges them into the local SQLite database. Conflict resolution favors server timestamps for historical records, but maintains local authority for active transactions.

12. Admin Dashboards

12.1 Attendance Dashboard Logic

The admin attendance dashboard is a read-only trust surface reflecting the operational reality of the library. It displays:

- Librarian identity and Device origin.
- Exact clock-in/clock-out timestamps.
- Sync status (to identify records not yet backed up).

Because records originate from the validated local clock-in (linked to a shift and HAV), administrators can trust that displayed attendance represents actual physical presence.

12.2 Reconciliation Logic

Reconciliation ensures local and remote states converge safely. The strategy is **Local-First, Server-Verified**. While the admin view reflects the merged truth, any discrepancies (such as a forced local record deletion) are flagged for audit rather than being silently auto-merged.

13. Security & Safety

13.1 Error Handling & Safety Guards

The system applies **fail-closed** behavior across all critical paths.

- **Guarded Operations:** Login, HAV, and Shift validation are "hard-guarded." If an error occurs during validation, the operation is terminated, and the user is prevented from proceeding.
- **Zero Uncertainty:** No transaction is recorded if the underlying data integrity check fails.

13.2 Security Considerations

Offline Threat Model & Mitigations

- **Device Compromise:** Mitigated by Device Binding; the account is useless on any other hardware.
- **Credential Reuse:** Mitigated by Shift Enforcement; access is revoked outside of library hours.
- **Replay Attempts:** Mitigated by the Commit-Based Sync Engine, which uses unique IDs to prevent re-processing of old data.

Privacy & Institutional Safety

- **Minimal Data Footprint:** The system avoids biometric and GPS tracking, relying instead on physical inventory (Book Scanning) for presence proof.
- **Tenant Isolation:** The **One-Institution-One-Database** model ensures that data from one library (e.g., Otona) can never leak into another (e.g., Fayda), even though they share the same application logic.

14. Implementation Report (Phase 1.0 → Phase 1.18.2)

14.1 Phase-by-Phase Summary

This section documents the incremental engineering evolution of the system from its initial foundation to its finalized Phase 1 release. Each phase was implemented, tested, and stabilized before proceeding to the next.

- **Phase 1.0 – Project Initialization**
 - Offline-first architecture established; Expo + React Native project scaffolded.
 - SQLite integration initialized and environment-based configuration introduced.

- **Phase 1.1 – Authentication Foundation**
 - Librarian and Admin role separation; Username + PIN authentication model.
 - Secure PIN hashing and session persistence layer implemented.
- **Phase 1.2 – Device Binding**
 - Unique device ID generation; Librarian account bound to a single device.
 - Unauthorized device access blocked at the login gate.
- **Phase 1.3 – Shift Management**
 - Shift scheduling model and time validation logic introduced.
 - Shift enforcement applied to all core librarian actions.
- **Phase 1.4 – Attendance Core**
 - shift_attendance schema finalized; Implicit clock-in logic implemented.
- **Phase 1.5 – Auto Logout Enforcement**
 - Shift-end auto logout scheduler; Session invalidation on shift expiration to prevent post-shift usage.
- **Phase 1.6 & 1.7 – Circulation (Borrow/Return)**
 - Book borrow/return lifecycles implemented with local-first transaction recording and stock restoration logic.
- **Phase 1.8 – Commit-Based Sync Engine**
 - Commit logging introduced to guarantee offline change persistence.
 - Sync-push and sync-pull workflows established.
- **Phase 1.9 – Fayda ID Integration**
 - Fayda ID scanning support added and standardized across identity flows.
- **Phase 1.10 – Admin Dashboard Foundations**
 - Attendance dashboard logic for read-only administrative oversight and local/synced data reconciliation.
- **Phase 1.11 & 1.12 – Hardening & Routing**
 - Fail-closed behavior enforced; Centralized scan routing architecture implemented for reusability.
- **Phase 1.13 – Transaction Safety**
 - Race condition handling for borrow/return and idempotent database updates.
- **Phase 1.14 – Shift-Aware Permissions**
 - Runtime shift validation added to mid-operation workflows to handle expirations during active tasks.
- **Phase 1.15 – Environment-Driven Configuration**
 - Standardized .env variables and feature toggles to control production behavior without code changes.

- **Phase 1.16 & 1.17 – Stability & Accuracy**
 - Date-based attendance filtering and sync retry resilience hardening.
- **Phase 1.18 – High Attendance Verification (HAV)**
 - Introduced book-scan-based verification for physical presence enforcement.
- **Phase 1.18.1 & 1.18.2 – HAV Refinement**
 - Login interception, mandatory verification gating, and duplicate scan prevention.

14.2 Major Design Decisions

- **Offline-First as a Core Constraint:** User workflows are never blocked by connectivity; local writes are the primary source of truth.
- **Device-Centric Trust:** Binding accounts to hardware eliminates the need for real-time remote session tracking and prevents credential sharing.
- **Physical Presence via Inventory:** Using book scans for attendance ensures librarians are on-site without requiring invasive GPS or specialized biometric hardware.
- **Commit-Based Sync:** Provides a deterministic audit trail and failure-resilient data reconciliation.

15. Maintainability & Future Phases

15.1 Extension Guidelines

Future development must adhere to these principles:

1. **Preserve Offline Guarantees:** Never introduce direct remote dependencies into core operational flows.
2. **Feature Flags:** Roll out new functionality via .env toggles to allow institutional customization.
3. **Commit Engine:** Any new table mutation must generate a local commit for the sync engine to ensure backup integrity.

15.2 Phase 2 Readiness

The system is architecturally prepared for:

- **Multi-branch Support:** Scaling to institutions with multiple physical locations.
- **Advanced Analytics:** Deep reporting on circulation trends and librarian efficiency.
- **Remote Configuration:** Moving .env policies to a remote-controlled central admin dashboard.

16. Conclusion

This document has presented a comprehensive technical specification, architectural overview, and implementation history of the **Offline-First Smart Library Management System**, developed and maintained by **DiguwaSoft** under the technical leadership of **Natinael Samuel** (Founder & Lead Engineer).

The system was intentionally engineered to operate under real-world institutional constraints, particularly in environments where network reliability, device availability, and operational discipline cannot be assumed. By adopting a device-centric, offline-first architecture, the platform ensures uninterrupted service continuity, predictable behavior, and strong data integrity guarantees regardless of connectivity conditions.

Throughout Phase 1.0 to Phase 1.18.2, the project evolved through a deliberate, incremental engineering process. Each phase reinforced foundational system principles—local authority, controlled synchronization, and operational trust—while introducing advanced functionality such as shift-enforced attendance, commit-based synchronization, and high-attendance verification through physical book scanning. These features were not added as isolated capabilities but as integrated components of a cohesive system design.

The decision to implement physical presence verification using book scans represents a notable innovation within the domain of library management systems. This approach avoids invasive surveillance technologies, respects institutional privacy expectations, and leverages existing physical assets, while still providing administrators with a high degree of confidence in attendance authenticity. Combined with strict shift enforcement and automatic logout mechanisms, the system achieves a balanced model of accountability and usability.

From a commercial standpoint, the platform has been architected to support scalable, multi-client deployment. The **one-institution-one-database model**, combined with DiguwaSoft's managed hosting and technical stewardship, enables flexible sales arrangements including per-institution licensing and subscription-based service agreements. This design minimizes cross-tenant risk while simplifying operational management and long-term maintenance.

This documentation serves both as:

1. **A Technical Reference:** For future developers, maintainers, and contractors.
2. **A Historical Engineering Record:** Capturing the rationale, structure, and execution of Phase 1 development.

As the system transitions toward future phases, the architectural foundations laid in Phase 1 provide a stable platform for expansion without fundamental redesign. Planned enhancements—such as advanced analytics, institutional reporting, and extended administrative capabilities—can be introduced within the existing framework while preserving offline resilience and data integrity.

In conclusion, the **Offline-First Smart Library Management System** stands as a production-ready, commercially viable, and technically robust solution, suitable for deployment across churches, schools, colleges, and public libraries. With its dual branding strategy—**Otona St. Mary Library Management App** and **Fayda Smart Library Management**—and centralized technical governance by DiguwaSoft, the system is well-positioned for sustainable growth, long-term support, and institutional trust.

Appendix

Appendix A: Environment Configuration (.env Variables)

The system relies on environment-level configuration to control behavior across deployments while keeping the codebase immutable. All environment variables are injected at build time.

A.1 Core Environment Variables

Variable Name	Type	Description
EXPO_PUBLIC_SUPABASE_URL	string	Supabase project URL for the institution
EXPO_PUBLIC_SUPABASE_ANON_KEY	string	Supabase anonymous access key
EXPO_PUBLIC_APP_NAME	string	Application display name (Otona / Fayda)
EXPO_PUBLIC_APP_VARIANT	string	Deployment variant identifier (church / education)

A.2 Attendance & Shift Control

Variable Name	Type	Default	Description
EXPO_PUBLIC_HIGH_ATTENDANCE_VERIFICATION	boolean	false	Enables High Attendance Verification (Phase 1.18)
EXPO_PUBLIC_NUMBER_OF_BOOKS_TO_SCAN	number	3	Required distinct books to verify attendance

EXPO_PUBLIC_SHIFT_ENFORCEMENT	boolean	true	Enforces shift boundaries for librarians
-------------------------------	---------	------	--

A.3 Branding & UI

Variable Name	Description
EXPO_PUBLIC_SPLASH_VARIANT	Selects splash screen asset bundle
EXPO_PUBLIC_LOGO_VARIANT	Selects logo bundle for the variant
EXPO_PUBLIC_PRIMARY_COLOR	Institution-specific UI theme color (Hex)

Appendix B: Database Tables (Reference Overview)

B.1 Key SQLite Tables

- **users:** fayda_id (PK), name, gender, phone, address, photo_uri.
- **librarians:** username (PK), role, pin_hash, pin_salt, device_id, require_pin_change.
- **books:** book_code (PK), title, author, copies.
- **transactions:** tx_id (PK), book_code, fayda_id, borrowed_at, returned_at, sync_status.
- **shift_attendance:** id, shift_id, username, clock_in_at, clock_out_at, verified.

Appendix C: Commit & Sync Infrastructure

C.1 commits Table

Column	Description
id	Auto-incrementing local ID
type	Mutation type: insert, update, delete
table_name	Target Supabase table
payload	JSON representation of the change
synced	Boolean status flag

C.2 Sync Flow Reference

1. Local mutation occurs → Commit record generated.
2. Commit marked as synced=false.
3. Sync engine attempts to push pending commits to Supabase in order.
4. Remote 200 OK received → Local commit marked synced=true.

Appendix D: Reference Files & Directories

D.1 Critical Application Paths

- /app/auth/: Authentication flows (Login, Change PIN).
- /app/attendance/: Attendance verification & scanning logic.
- /db/: SQLite access layer and domain-specific queries.
- /lib/: Core logic for security, session, and shift management.
- /utils/: Shared guards and validation utilities.

Appendix E: Terminology Reference

- **Offline-First:** The local device database is the authoritative runtime source.
- **Commit:** An atomic record of a local change queued for synchronization.
- **Shift:** A time-bounded work session required for librarian operations.
- **HAV:** High Attendance Verification; proving presence via physical item scans.
- **Device Binding:** Restricted account usage to a single verified hardware ID.

Appendix F: Ownership & Maintenance

- **System Owner:** DiguwaSoft
- **Technical Authority:** Natinael Samuel
- **Deployment Model:** Fully Managed by DiguwaSoft
- **Tenant Strategy:** One Supabase DB per institution
- **Source Control:** Private Repository